YILDIZ TECHNICAL UNIVERSITY
MATHEMATICAL ENGINEERING DEPARTMENT

PROJECT FOR THE INTRODUCTION TO ARTIFICIAL INTELLIGENCE (2026 SPRING)

HANOI TOWER
SOLUTIONS WITH AI SEARCH ALGORITHMS

GROUP MEMBERS:
-Alperen İlhan İPÇİ
-Bensu KOCATEPE
-Ezgi Nur EREN
-Gökhan KESKİN
-Hamide OKUMUŞ
-Selma CEMAL

MAY 2026

CONTENTS

CONTENTS **2**
1.INTRODUCTION **3**
1.1.WHAT IS HANOI TOWER? 3
1.2.Why Hanoi Tower is Classified as an Artificial Intelligence Search Problem? 3
2.PROBLEM DEFINITION **4**
2.1.States 4
2.2.Initial State Generation (Root Node): 4
2.3.Goal State Generation (Target Node): 4
2.4.Legal Actions and Constraints 5
2.5.Transition Model 6
3.HEURISTIC AND ALGORITHM DESIGN **7**
3.1.Implementation of the Generalized A*Search Algorithm 7
3.2.Heuristic Function (h(n)) Selection and Admissibility Analysis 9
3.3.Code Architecture and Modified Source Files 9
4.SIMULATION RESULTS AND PERFORMANCE ANALYSIS **10**
4.1.Quantitative Evaluation and Simulation Metrics Table 10
4.2. Experimental Verification via Terminal Outputs 13
4.2.1. 3-Disk Problem Scale Execution: 13
5.CONCLUSION **14**

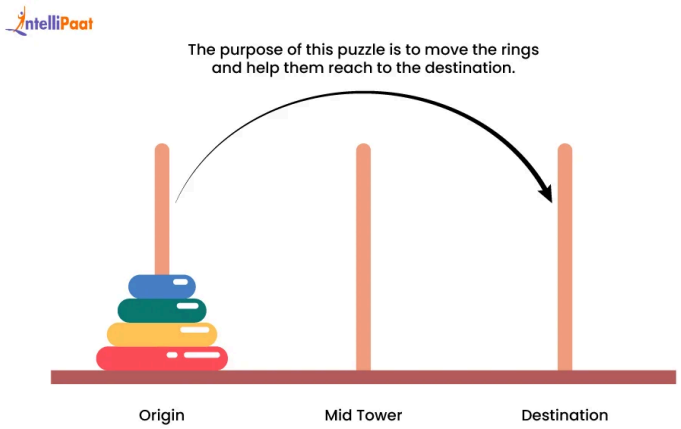
1.INTRODUCTION

1.1.WHAT IS HANOI TOWER?

The Tower of Hanoi is a puzzle invented by French mathematician Édouard Lucas in 1883. The setup of the game is quite simple: there are 3 pegs standing side by side, and initially, a number of *N* disks are stacked on one of these pegs from largest to smallest (forming a conical shape).

The game has a single objective: to move all the disks from the starting peg to the target peg while adhering to 3 golden rules. These rules are as follows:

- Only one disk can be moved at a time.
- Only the top disk of a peg can be moved.
- A larger disk can absolutely never be placed on top of a smaller disk.



1.2.Why Hanoi Tower is Classified as an Artificial Intelligence Search Problem?

The Tower of Hanoi represents a classic sequential decision-making and pathfinding problem for an artificial intelligence agent. Rather than employing a closed-form mathematical formula, the agent must autonomously navigate a structured state space governed by an initial configuration, operational constraints, and a well-defined goal state. As the puzzle scales from 3 to 5, and ultimately to 9 disks, the number of distinct permutations within the state space grows exponentially according to the formula 3^N. This phenomenon, known as a combinatorial explosion, makes it highly complex; without an intelligent traversal strategy, an uninformed agent would easily succumb to this vast environment and become trapped in infinite, sub-optimal loops.

Consequently, the puzzle serves as an ideal benchmark for evaluating informed search paradigms. Instead of relying on blind, brute-force exploration, the AI agent utilizes a well-designed heuristic function (h(n)) within its transition model to dynamically evaluate valid actions. This allows the system to intelligently prune suboptimal branches of the search tree, eliminate cyclical redundancies, and efficiently isolate the shortest, most optimal path to the goal configuration.

2.PROBLEM DEFINITION

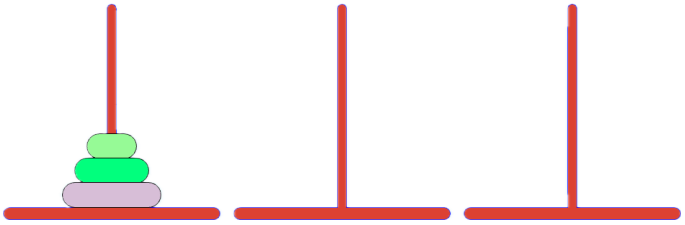
2.1.States

State structure is defined with three components: Integer variable num_disks for the preferred number of disks in the current problem instance; an integer array disk[MAX_DISKS] where the index indicates the specific disk and its value represents the peg it is currently located on (0=A, 1=B, 2=C); and a float variable h_n storing the heuristic evaluation value, which is used in A* and Greedy search algorithms.

```
typedef struct State
{
    int num_disks;
    int disk[MAX_DISKS]; // disk[i] = peg index of disk i (0 = A, 1 = B, 2 = C)
    float h_n;          // Heuristic estimate to goal
} State;
```

2.2.Initial State Generation (Root Node):

During its first invocation (call_count=1), the function initiates the environment setup. It prompts the user to define the scale of the problem by selecting the number of active disks (N = 3, 5 or 9). After validating the input and dynamically allocating memory for the state structure, a for loop is utilized to assign the value 0 to every element in the disk array. This mathematical assignment ensures that all disks are initially placed on the designated source peg, Peg A.

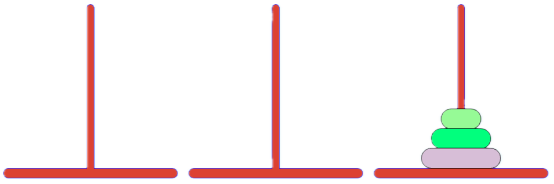


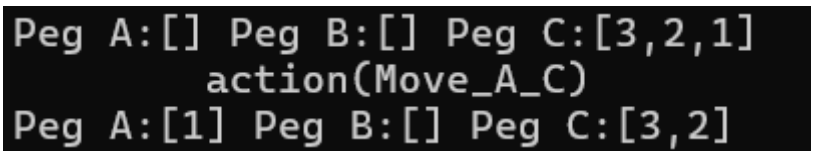
```
Peg A:[3,2,1] Peg B:[] Peg C:[]
```

Output of the algorithm starting with the initial state.

2.3.Goal State Generation (Target Node):

When the function is called subsequently to define the target configuration, the else block is triggered. Bypassing the user input phase, it reuses the retained disk_count variable. The loop then systematically assigns the value 2 to all array elements. This effectively models the required terminal criteria, placing all N disks onto the destination peg, Peg C.





The algorithm ends when the last disk on peg A moves to peg C; reaching the goal state where peg C is full and the other pegs are completely empty.

```
State* Create_State()
{
    static int disk_count = 0;
    static int call_count = 0;
    State *state = (State*)malloc(sizeof(State));
    if(state == NULL)
        Warning_Memory_Allocation();

    call_count++;
    if(call_count == 1){
        do{
            printf("Enter the number of disks (3, 5 or 9): ");
            scanf("%d", &disk_count);
        } while(disk_count != 3 && disk_count != 5 && disk_count != 9);

        state->num_disks = disk_count;
        for(int i = 0; i < state->num_disks; i++)
            state->disk[i] = 0; // all disks start on peg A
    }
    else{
        state->num_disks = disk_count;
        for(int i = 0; i < state->num_disks; i++)
            state->disk[i] = 2; // goal is all disks on peg C
    }

    state->h_n = 0.0f;
    return state;
}
```

2.4.Legal Actions and Constraints

In a state-space search paradigm, an action is a valid operator that transitions the environment from a current state s to a successor state s'. For the Tower of Hanoi, an action is defined as relocating a single disk from a source peg to a destination peg.

To ensure state-space validity during the generalized A* search execution, every generated action must strictly satisfy the following operational constraints:

- Only the topmost, unobstructed disk of the source peg can be legally moved during a transition.
- The agent is restricted to moving exactly one disk per state transition, meaning that the simultaneous movement of multiple disks is strictly prohibited.
- A larger disk cannot be placed on top of a smaller disk; thus, the destination peg must either be completely vacant or host a topmost disk with a diameter strictly greater than the moving disk.

Any transition violating these constraints is automatically filtered out by the problem-specific validation logic within SpecificToProblem.c, ensuring the search tree expands only along valid paths.

```
enum ACTIONS // All possible actions for Tower of Hanoi
{
    Move_A_B, Move_A_C,
    Move_B_A, Move_B_C,
    Move_C_A, Move_C_B
};
```

2.5.Transition Model

```
int Result(const State *const parent_state, const enum ACTIONS action,
    Transition_Model *const trans_model)
{
    int source, target;
    if(!action_source_target(action, &source, &target))
        return FALSE;

    int source_top = top_disk_on_peg(parent_state, source);
    if(source_top == -1)
        return FALSE; // no disk to move

    int target_top = top_disk_on_peg(parent_state, target);
    if(target_top != -1 && source_top > target_top)
        return FALSE; // cannot place larger disk on smaller one

    State new_state = *parent_state;
    new_state.disk[source_top] = target;
    trans_model->new_state = new_state;
    trans_model->step_cost = 1.0f;
    return TRUE;
}
```

In a state space search paradigm, the transition model defines the logical rules for moving from a parent state to a successor state. In our codebase, this transition logic is handled by the result function alongside the Transition_Model structure. When an action is proposed, the system first checks two logical constraints: it verifies that the source peg is not empty, and it strictly ensures that a larger disk is never placed on top of a smaller disk. If these conditions fail, the transition is logically invalid and returns false. If the action is legal, the model mathematically updates the state by assigning the moved disk's array index to the new target peg. This successful configuration is saved in the new_state variable of the Transition_Model struct, and the system assigns a uniform step_cost of 1.0, which incrementally updates the path cost g(n).

3.HEURISTIC AND ALGORITHM DESIGN

3.1.Implementation of the Generalized A*Search Algorithm

In state space search problems, uninformed search algorithms explore the environment without any domain specific knowledge. These algorithms determine the expansion of the search tree relying solely on either the depth of the node or the cumulative path cost, g(n). While complete and optimal under certain conditions, uninformed structures suffer from severe inefficiencies and exponential time complexities because they expand uniformly in all directions.

To transform this blind structure into a highly efficient, informed search strategy, problem specific intelligence must be injected into the transition model. This is achieved through a heuristic function, h(n), which estimates the remaining cost from the current state to the goal. In this project, we implement a Generalized A* Search, which introduces an adjustable weighting parameter, alpha. Consequently, the strict evaluation function that dictates the frontier expansion is defined as: $f(n) = g(n) + alpha*h(n)$

```
case GeneralizedAStarSearch:
    child->state.h_n = Compute_Heuristic_Function(&(child->state), goal_state);
    if(temp_node!=NULL){
        float child_f = child->path_cost + (alpha * child->state.h_n);
        float temp_f = temp_node->path_cost + (alpha * temp_node->state.h_n);

        if(child_f < temp_f)
            Remove_Node_From_Frontier(temp_node, &frontier);
        else
            continue;
    }
    Insert_Priority_Queue_GENERALIZED_A_Star(child, &frontier, alpha);
    break;
```

Pic3.1.1

As illustrated in Pic 3.1.1, the algorithmic transition form a blind structure to the Generalized A* Search requires dynamic evaluation of both the historical path cost and the heuristic projection. When a new child node is generated, the algorithm first computes its heuristic estimate via the Compute_Heuristic_Function. If this generated state already exists in the frontier (temp_node != NULL), the system calculates their respective generalized evaluation values (child_f and temp_f). If the newly discovered path yields a strictly lower f(n) value (child_f < temp_f), the suboptimal older node is pruned from the memory (Remove_Node_From_Frontier), and the superior child continues. Finally, the Insert_Priority_Queue_GENERALIZED_A_Star function enqueues the node, ensuring the frontier remains strictly ordered by the lowest f(n) value rather than blindly expanding the shallowest nodes.

```
void Insert_Priority_Queue_GENERALIZED_A_Star(Node *const child, Queue **frontier, float alpha)
{
    Queue *temp_queue;
    Queue *new_queue = (Queue*)malloc(sizeof(Queue));
    if(new_queue==NULL)
        Warning_Memory_Allocation();

    new_queue->node = child;

    if(Empty(*frontier)){
        new_queue->next = NULL;
        *frontier = new_queue;
    }
    else{
        float child_f = child->path_cost + (alpha * child->state.h_n);
        float first_frontier_f = (*frontier)->node->path_cost + (alpha * (*frontier)->node->state.h_n);

        if(child_f < first_frontier_f) {
            new_queue->next = *frontier;
            *frontier = new_queue;
        }
        else{
            for(temp_queue = *frontier; temp_queue->next != NULL; temp_queue = temp_queue->next){
                float next_frontier_f = temp_queue->next->node->path_cost + (alpha * temp_queue->next->node->state.h_n);

                if(child_f < next_frontier_f){
                     new_queue->next = temp_queue->next;
                     temp_queue->next = new_queue;
                     return;
                }
            }
            temp_queue->next = new_queue;
            new_queue->next = NULL;
        }
    }
}
```

Pic 3.1.2

As shown in Pic 3.1.2, the Insert_Priority_Queue_GENERALIZED_A_Star function is responsible for maintaining the frontier as an ordered priority queue based on the generalized cost evaluation function, f(n) = g(n) + alpha*h(n).

When a new valid child node is generated, the function dynamically allocates memory for a new queue element. If the frontier is currently populated, the system calculates the evaluation value of the incoming node (child_f) by summing its cumulative path cost (path_cost, representing g(n)) and its alpha weighted heuristic estimate (alpha * state.h_n). The function then sequentially traverses the existing frontier to determine the exact insertion point. By dynamically comparing child_f against the evaluated costs of the nodes already residing in the queue (next_frontier_f), it inserts the new node precisely where its evaluation cost is strictly less than the subsequent node's cost. This localized sorting mechanism ensures that the frontier remains strictly in ascending order, guaranteeing that the search engine always pops and expands the most promising node with the lowest generalized cost first.

3.2.Heuristic Function (h(n)) Selection and Admissibility Analysis

```
float Compute_Heuristic_Function(const State *const state, const State *const goal)
{
    int misplaced = 0;
    for(int i = 0; i < state->num_disks; i++){
        if(state->disk[i] != goal->disk[i])
            misplaced += (1 << i);
    }
    return (float)misplaced;
}
```

To guide the search engine efficiently through the exponential state space, an advanced admissible heuristic function h(n) is formulated in SpecificToProblem.c. Rather than simply counting the number of misplaced disks, the implemented function uses a bitwise weight distribution to reflect the inherent sequential constraints of the puzzle. Each disk *i* that is not currently located on the destination peg contributes an exponential weight of 2^i to the total heuristic evaluation. Formally, this is expressed as:

$$h(n) = \sum_{i \in \text{misplaced}} 2^i$$

Admissibility Analysis: A heuristic is strictly admissible if it never overestimates the actual remaining cost to reach the goal (h(n) <= h*(n)). In the Tower of Hanoi, for any disk *i* to be legally transferred to its goal peg, all smaller disks (0 to i-1) must first be cleared and stacked on the alternative auxiliary peg. This structural recursion guarantees that the true number of minimum required actions h*(n) always aligns with or exceeds the 2^i geometric progression. Thus, h(n) is mathematically optimistic, ensuring that the generalized A* framework remains globally optimal.

3.3.Code Architecture and Modified Source Files

The foundational codebase provided for the search framework was originally structured for a geographic pathfinding domain, where distinct states were uniquely identified by an integer index representing a city (state → city). To adapt this architecture to the Tower of Hanoi problem, substantial modifications were executed within HashTable.c, specifically inside the Generate_HashTable_Key function.

In the original implementation, the unique hash key was generated by extracting the digits of the city integer. However, a state in the Tower of Hanoi is defined by the exact positioning of all active disks across the three pegs.Therefore, we replaced the legacy city-based allocation with a dynamic loop that iterates through state –> num_disks. The position of each individual disk (state →disk[i]) is converted into a character and stored sequentially within the key character array, which is then explicitly null-terminated. This structural modification guarantees that every unique permutation of disks yields a completely distinct hash key, enabling the explorer_set to correctly identify visited states and prune cyclical paths during the graph search.

```
void Generate_HashTable_Key(const State *const state, unsigned char* key)
{
    int i;
    for(i = 0; i < state->num_disks; i++){
        key[i] = (unsigned char)('0' + state->disk[i]);
    }
    key[state->num_disks] = '\0';

    if(state->num_disks + 1 > MAX_KEY_SIZE){
        printf("ERROR: MAX_KEY_SIZE is exceeded in Generate_HashTable_Key. \n");
        exit(-1);
    }
}
```

4.SIMULATION RESULTS AND PERFORMANCE ANALYSIS

4.1.Quantitative Evaluation and Simulation Metrics Table

Problem Size: 3

Algorithm Mode	Optimal Path Cost (Moves)	Path Cost	Searched Nodes	Generated Nodes	Computational Time (ms)
Breadth-First Search (BFS)	**7**	7	25	53	~ 0
Depth-First Search (DFS)	**7**	9	15	26	~ 0
Uniform-Cost Search (UCS)	**7**	7	25	71	~ 0
Iterative Deepening Search (IDS)	**7**	7	71	131	~ 0
Depth-Limited Search (DLS)	**7**	Limit 5→Solution can not be found Limit 7→7 Limit 15→9	Limit 5→Solution can not be found Limit 7→15 Limit 15→15	Limit 5→Solution can not be found Limit 7→26 Limit 15→26	~ 0
Greedy Best-First Search	**7**	7	23	50	~ 0
Standard A* Search	**7**	7	14	39	~ 0
Generalized A* Search	**7**	alpha=0 →7 alpha=0 .5→7	alpha=0 → 25 alpha=0.5→16	alpha=0→ 71 alpha=0.5→45	~ 0

		alpha=1→7 alpha=5→7	alpha=1→15 alpha=5→18	alpha=1→42 alpha=5→51	

Problem Size: 5

Algorithm Mode	Optimal Path Cost (Moves)	Path Cost	Searched Nodes	Generated Nodes	Computational Time (ms)
Breadth-First Search (BFS)	**31**	31	233	617	22
Depth-First Search (DFS)	**31**	81	123	242	48
Uniform-Cost Search (UCS)	**31**	31	233	695	17,4
Iterative Deepening Search (IDS)	**31**	45	3566	9402	30
Depth-Limited Search (DLS)	**31**	Limit 20→The solution can not be found Limit 31→ The solution can not be found Limit 63→47 Limit 100→81 Limit 150→ 81	Limit 20→ The solution can not be found Limit 31→ The solution can not be found Limit 63→132 Limit 100→123 Limit 150→123	Limit 20→ The solution can not be found Limit 31→ The solution can not be found Limit 63→299 Limit 100→242 Limit 150→242	Limit 63→32 Limit 100→46 Limit 150→50,7
Greedy Best-First Search	**31**	35	181	500	20,4
Standard A* Search	**31**	31	150	447	17
Generalized A* Search	**31**	alpha=0→31 alpha=0.5→31 alpha=1→31 alpha=5→31	alpha=0→233 alpha=0.5→136 alpha=1→153 alpha=5→162	alpha=0→695 alpha=0.5→405 alpha=1→456 alpha=5→483	alpha=0→ 13,7 alpha=0.5 →26 alpha=1→ 19 alpha=5→ 17,2

Problem Size: 9

Algorithm Mode	Optimal Path Cost (Moves)	Path Cost	Searched Nodes	Generated Nodes	Computational Time (ms)
Breadth-First Search (BFS)	**511**	511	19513	57257	516
Depth-First Search (DFS)	**511**	6561	9843	19682	6751
Uniform-Cost Search (UCS)	**511**	511	19513	58535	471
Iterative Deepening Search (IDS)	**511**	TIMEOUT	TIMEOUT	TIMEOUT	TIMEOUT
Depth-Limited Search (DLS)	**511**	Limit 400→The solution can not be found Limit 511→The solution can not be found Limit 1023→The solution can not be found Limit 2047→The solution can not be found Limit 3000→The solution can not be found Limit 3500→3325	Limit 400→The solution can not be found Limit 511→The solution can not be found Limit 1023→The solution can not be found Limit 2047→The solution can not be found Limit 3000→The solution can not be found Limit 3500→7423	Limit 400→The solution can not be found Limit 511→The solution can not be found Limit 1023→The solution can not be found Limit 2047→The solution can not be found Limit 3000→The solution can not be found Limit 3500→17123	Limit 3500→3070,3
Greedy Best-First Search	**511**	667	14081	42056	647,3
Standard A* Search	**511**	511	13062	39183	488
Generalized A* Search	**511**	alpha=0→511 alpha=0.5→511	alpha=0→19513 alpha=0.5→10936	alpha=0→58535 alpha=0.5→3280	alpha=0→517

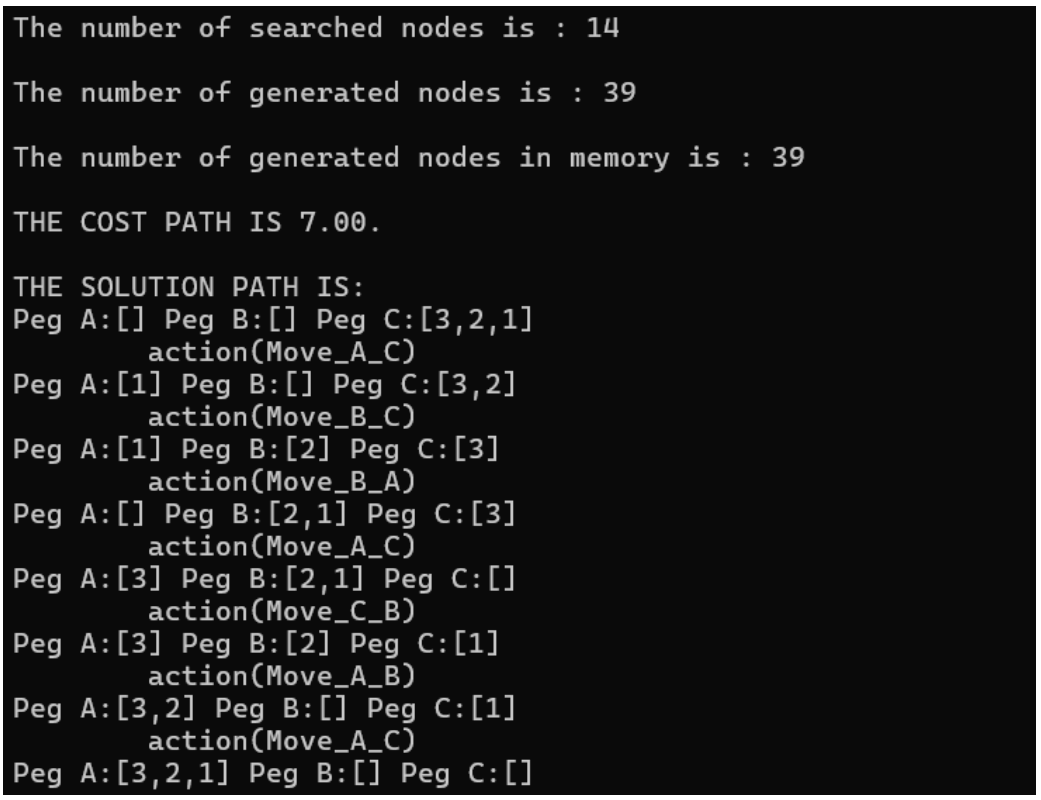
		alpha=1→511 alpha=5→511	alpha=1→13077 alpha=5→13122	5 alpha=1→39228 alpha=5→39363	alpha=0.5 →507,3 alpha=1→4 65 alpha=5→4 65,5

4.2. Experimental Verification via Terminal Outputs

4.2.1. 3-Disk Problem Scale Execution:

```
The number of searched nodes is : 25

The number of generated nodes is : 53

The number of generated nodes in memory is : 53

THE COST PATH IS 7.00.

THE SOLUTION PATH IS:
Peg A:[] Peg B:[] Peg C:[3,2,1]
        action(Move_A_C)
Peg A:[1] Peg B:[] Peg C:[3,2]
        action(Move_B_C)
Peg A:[1] Peg B:[2] Peg C:[3]
        action(Move_B_A)
Peg A:[] Peg B:[2,1] Peg C:[3]
        action(Move_A_C)
Peg A:[3] Peg B:[2,1] Peg C:[]
        action(Move_C_B)
Peg A:[3] Peg B:[2] Peg C:[1]
        action(Move_A_B)
Peg A:[3,2] Peg B:[] Peg C:[1]
        action(Move_A_C)
Peg A:[3,2,1] Peg B:[] Peg C:[]
```

Breadth First Search for 3 disks



A* Search for 3 disks

4.3.Comparative Performance and State-Space Evaluation

Terminal outputs show that execution time and node generation increase exponentially across the 3,5 and 9-disk configurations. This happens because the state space grows at a rate of 3^N,while the shortest path follows 2^N-1. As a result, uninformed search algorithms explore too many nodes, which consumes a lot of system memory. To prevent infinite recursion, the algorithms treat the environment as a graph using a hash table. Each new state is hashed into a unique key and stored. Before trying a new move,the transition model checks this table;if the state already exist, it prunes that branch. This prevents the agent from looping through the same states and keeps the search process manageable despite the high number of possibilities.

5.CONCLUSION

The terminal outputs demonstrate that the Generalized A* algorithm provides a clear advantage in the time and memory management compared to blind search algorithms. Since blind searches lack a heuristic guide, they expand the search tree in every direction. As the number of disks increases, this undirected expansion causes an exponential growth in the generated nodes, leading to rapid memory consumption and long execution times.

Generalized A* solves these bottlenecks by utilizing a heuristic function. Instead of checking every possible move equally, the algorithm evaluates the generated states according to their distance to the target and prioritizes the most promising paths. This approach allows the

transition model to prune unnecessary branches early in the process. As a result, the total number of nodes kept in memory is minimized, and the algorithm successfully reaches the optimal configuration without the memory exhausting and temporal delays seen in uninformed searches.